

THE SS138 PROTOCOL: A DETERMINISTIC FINITE-HORIZON EDGE ESTIMATOR FOR MULTI- VARIABLE STATE-SPACE TRAJECTORY SYNC

Technical Specification White Paper

Author: SquirrelSniper138

Repository: [github.com/](https://github.com/SquirrelSniper/SS138-Protocol)

[SquirrelSniper/SS138-Protocol](https://github.com/SquirrelSniper/SS138-Protocol)

Abstract

This paper introduces the SS138 protocol, a specialized finite-horizon edge estimator designed to isolate systemic tracking drift, packet erasures, and multi-variable state anomalies before data propagates to downstream execution layers. Traditional distributed networks

face compounding synchronization errors from cumulative numerical rounding, network jitter, and uncoordinated endpoint sampling clocks, typically requiring computationally expensive consensus routines at the database layer to retroactively reconcile errors. The SS138 architecture mitigates this bottleneck by deploying a proactive, deterministic state estimation gateway at the network ingestion boundary. The protocol processes incoming continuous streams within a discrete, 12-second Macro Horizon divided into four consecutive, 3-second localized processing phases. When network

degradation or partial packet drops occur, a discrete tri-state software logic abstraction (+1, 0, -1) instantly assigns an erasure flag (-1) to the degraded registers, preserving state-machine dimension alignment without inducing immediate, latency-heavy transport-layer retransmission loops.

At the terminal boundary of the observation horizon, the gateway executes a constrained, multi-variable quadratic optimization routine. By utilizing the verified states of adjacent healthy phases as hard spatial boundary conditions and enforcing an inequality acceleration threshold ($\leq$

Lambda_max), the system minimizes total kinetic strain variance across the window. This convex optimization constraint collapses the open linear degrees of freedom down to a singular, mathematically unique reconstructed trajectory with minimal computational overhead. Validated through numerical simulation, the finalized state vector is mapped via a static canonical projection matrix ($P_{12 \rightarrow 3}$) to a 3-dimensional physical execution register. This architecture achieves real-time boundary verification and deterministic input sanitization, hardening the ingestion interface

against external volatility and ensuring an unalterable state baseline for downstream distributed ledger systems.

1. Introduction: Objectives & State-Space Methodology

Modern distributed information networks processing high-frequency continuous tracking data routinely encounter compounding state synchronization errors. These anomalies originate from cumulative numerical floating-point rounding errors, transport-layer network jitter, and uncoordinated endpoint sampling clocks. Left unchecked, these minor localized variations cascade across

connected consensus nodes, culminating in network state divergence, database desynchronization, and transactional processing failures. Traditional recovery mechanisms deploy reactive verification and consensus routines at the central database layer, demanding immense computational and network overhead to retroactively audit and reconcile basic edge errors.

The SS138 protocol mitigates this systemic bottleneck by introducing a proactive, deterministic state estimation gateway at the absolute boundary of network ingestion.

Synthesized via multi-variable optimal control principles, this architecture enforces geometric and kinematic constraints directly on incoming time-series data streams prior to downstream execution. Rather than treating ingestion as an arbitrary, uncorrelated packet queue, the protocol models the incoming data stream as a continuous, highly correlated trajectory traversing a multi-dimensional state space. By binding high-throughput tracking inputs to strict, finite temporal horizons and deterministic spatial matrices, the edge layer achieves real-time boundary verification. Any input vector that fails to satisfy

these localized boundary conditions is systematically isolated at the ingestion point, providing downstream systems with a guaranteed, verified, and unalterable state baseline.

2. Ingestion Infrastructure and Gateway Specifications

The SS138 gateway functions as a rigid state-machine boundary that filters, structures, and validates incoming data payloads. The physical and logical architecture is structured into three core operational layers: Discrete Finite-Horizon Observation, Tri-State Erasure Flagging, and Deterministic Boundary Reconstruction.

2.1 Finite-Horizon Observation Configuration

The application-layer pipeline maps continuous data streams directly onto a structured temporal index. The 12-second Macro Horizon functions as a discrete auditing window, segmented into four consecutive, 3-second localized processing phases. Incoming metrics are cached sequentially: Phase 1 captures seconds 0 through 3; Phase 2 captures seconds 3 through 6; Phase 3 captures seconds 6 through 9; and Phase 4 handles the terminal window from seconds 9 through 12. This configuration decouples active

ingestion from the core processing queue, effectively buffering network jitter and decoupling the computationally intensive verification loop from the primary data transport stream to ensure real-time deterministic processing.

2.2 Tri-State Logic Matrix Execution

The state-machine handles data validation using a discrete register flag array rather than unconstrained binary streams. When network degradation, physical interference, or partial packet drops occur, traditional binary ingestion systems struggle with empty null-pointers, causing buffer errors, tracking timeouts, or forcing continuous,

latency-heavy retransmission requests. The SS138 architecture bypasses this limitation by deploying a discrete tri-state logic abstraction directly within the software execution and register layer, where system states are mapped explicitly to one of three structural values:

State Logic Positive One (+1):

Assigned to a phase register when the incoming metrics perfectly satisfy expected baseline thresholds.

State Logic Zero (0): Assigned to a phase register during nominal system standby, idle states, or inactive transmission frames.

State Logic Negative One (-1):
Assigned instantly when a phase window detects missing packets, corruption, or out-of-bounds metrics. By logging a corrupted window as an explicit structural deficit (-1) rather than dropping the packet or throwing a system exception, the gateway preserves register synchronization and maintains the exact dimensional integrity of the state array. The system pipeline never stalls; the missing indices are cleanly tracked as known structural variables to be resolved at the horizon boundary.

2.3 Kinematic Boundary Smoothing (Boundary-Value Reconstruction)

To achieve zero-ambiguity data integrity across lossy channels without demanding extra transmission bandwidth, the gateway deploys a Finite-Horizon Smoothing Filter. When a continuous tracking trajectory encounters localized drops—such as a critical drop during Phase 2 (3–6s) and Phase 3 (6–9s)—the gateway utilizes the verified, healthy states captured during Phase 1 (0–3s) and Phase 4 (9–12s) as rigid, hard spatial boundary conditions. At the terminal 12th second of the Macro Horizon, the system runs a multi-variable optimization routine. Rather than guessing data or relying

on arbitrary linear interpolation, the estimator calculates the unique state trajectory across the missing phases by executing a convex kinetic minimization transform. Because the underlying physical or systemic process being tracked is governed by known second-order limits, the algorithm solves for the smoothest possible curve (minimizing total acceleration variance) that perfectly connects the known boundary vectors. This mathematical constraint collapses the linear system's degrees of freedom, yielding a singular, deterministic trajectory reconstruction with minimum

computational overhead.

3. Mathematical Optimization and Mapping Logic

3.1 Spatial Mapping: Multi-Dimensional Manifold Projection

The mathematical foundation of the protocol replaces dynamic, unconstrained floating-point calculations with an unchanging, structured coordinate matrix.

Incoming data payloads are mapped as multi-variable state vectors (S_k) within a 4-dimensional hyper-coordinate system ($d = 4$), structured as a column matrix:

$$S_k = [x_{k,1} ; x_{k,2} ; x_{k,3} ; x_{k,4}]$$

This 4-dimensional vector space provides the precise integer basis

required to natively span the tracking coordinate system without accumulating periodic rounding errors or floating-point unit (FPU) noise over extended continuous runtimes.

3.2 Convex Optimization and Kinetic Strain Minimization

During an erasure event where data from Phase 2 and Phase 3 are lost, the system faces 12 unknown scalar variables (3 dropped steps multiplied by 4 vector components). To resolve this underdetermined search space without ambiguity, the gateway executes a localized quadratic optimization routine at the terminal 12th second.

The objective function minimizes the sum of the squared second-order differences across the entire 12-second observation window:

Minimize: Sum from $k=1$ to 10 of $\| (\Delta^2 * S_k) / (\Delta t^2) \|^2$

Because this objective function is strictly quadratic, the optimization path is completely convex. When bounded by the hard coordinate inputs of Phase 1 and Phase 4, the optimization vise collapses the 9 open linear degrees of freedom down to exactly zero spatial ambiguity, converging on the single, unique global minimum representing the minimum-acceleration curve across the gap.

3.3 Boundary-Value Bounding Envelopes

To prevent the optimization engine from projecting erratic or unbounded solutions, the optimization loop is continuously locked inside a rigid box-constraint boundary envelope regularized by the tri-state logic parameters:

$$\| S_{\text{candidate}} \| \leq \text{Lambda_max}$$

This constraint ensures that all reconstructed trajectories remain strictly within the physical and systemic limits of the tracking plant, rejecting localized network entropy and preventing malformed, out-of-bounds states from ever propagating past the gateway

interface.

3.4 Canonical Dimension Reduction

Once the 4-dimensional state manifold is completely reconstructed and verified at the terminal 12th second, it passes through a static canonical projection matrix ($P_{12 \rightarrow 3}$) to map the data down to a 3-dimensional physical execution register ($d = 3$). The matrix transformation is defined as follows, where α equals $\pi / 6$:

Row 1: [$\cos(0\alpha)$, $\cos(1\alpha)$, $\cos(2\alpha)$, $\cos(3\alpha)$]

Row 2: [$\sin(0\alpha)$, $\sin(1\alpha)$, $\sin(2\alpha)$, $\sin(3\alpha)$]

Row 3: [1.0 , 1.0 , 1.0 , 1.0]

This linear reduction condenses the validated tracking data into a compact, standardized 3D matrix, ensuring optimized processing speeds for downstream ledger storage and core backend nodes.

4. Formal Technical System Claims

Claim 1: A Deterministic Ingress Method for Multi-Variable State-Space Estimation

A method for isolating systemic baseline drift and filtering transmission noise from a continuous multi-variable tracking stream at an application-layer edge gateway interface, the method comprising the steps of:
Establishing an edge gateway

execution environment configured to act as a deterministic ingestion interface;

Receiving a continuous time-series data payload at said interface representing a continuous state-space trajectory;

Projecting said data payload onto a 4-dimensional hyper-coordinate matrix to establish an integer-based tracking baseline;

Executing a multi-variable optimization loop at a terminal observation boundary to reconstruct missing states via kinetic continuity constraints; and

Projecting the finalized, synchronized state vector down to a

3-dimensional physical execution register, thereby preventing the propagation of drifted or corrupted states to a downstream database network.

Claim 2: The Finite-Horizon Discrete Observation Horizon

The method of Claim 1, wherein governing the temporal boundaries of the data ingestion interface comprises:

Configuring a strict finite-horizon observation window at a base interval of twelve seconds;

Segmenting said twelve-second macro horizon into four independent, consecutive, three-second localized processing

phases; and

Batching incoming data packets exclusively within each localized three-second phase, thereby asynchronously isolating the active verification cycle from the main network transport stream.

Claim 3: Tri-State Erasure Flagging and Pipeline Synchronization

The method of Claim 1, wherein managing packet drops and signal degradation without introducing pipeline latency comprises:

Monitoring the localized processing phases for missing, corrupted, or out-of-bounds data packets;

Instantly assigning a discrete tri-state erasure flag (-1) to the register

indices of any degraded phase window; and

Maintaining absolute state-machine dimension alignment across the ingestion array using said erasure flags, thereby bypassing immediate transport-layer retransmission loops and preserving processing throughput.

Claim 4: Bounded Convex Trajectory Reconstruction

The method of Claim 1, wherein resolving underdetermined tracking states during localized phase erasures comprises:

Capturing the verified state vectors of Phase 1 and Phase 4 to serve as rigid, unalterable boundary

conditions;

Formulating a strictly convex objective function that calculates the sum of squared second-order differences across the observation horizon; and

Executing a constrained numerical optimization loop to minimize total kinetic acceleration variance, thereby collapsing the remaining degrees of freedom down to a singular, mathematically unique reconstructed trajectory.

Claim 5: Single-Node Boundary

Hardening and Secure Logging

The method of Claim 1, wherein hardening the gateway interface against input manipulation and

environmental anomalies comprises:
Bounding the optimization search space within a strict, predefined maximum acceleration envelope ($\leq \text{Lambda_max}$);
Committing the fully reconstructed, synchronized 3D execution matrix into a secure state-save layer upon the close of the 12th second; and
Generating a unique, deterministic cryptographic memory signature from the active state baseline, reducing the available attack surface and providing an unalterable, audited ingestion reference for downstream ledger components.